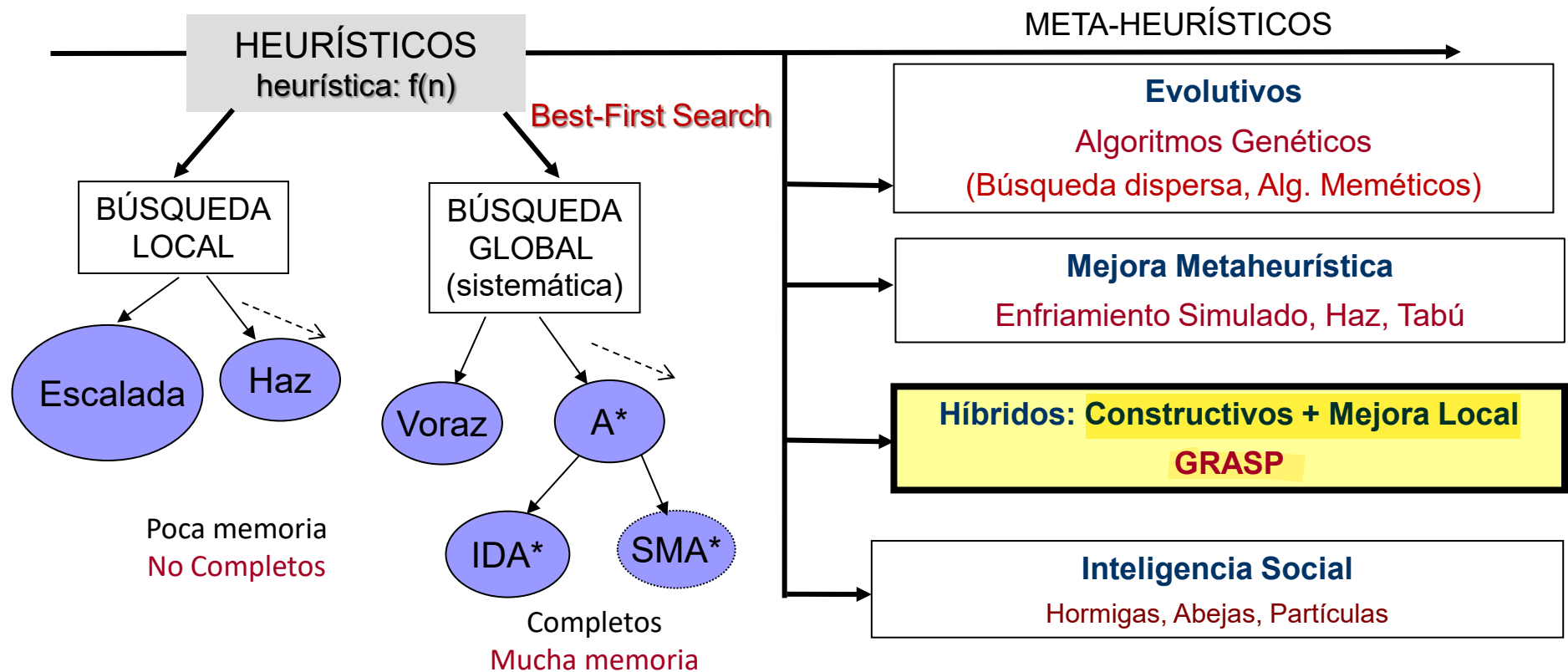


Tema 4: Metaheurísticas Constructivas (+ mejora local)

GRASP

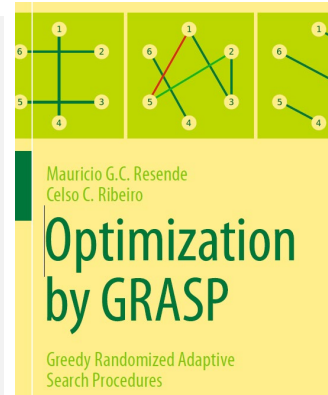
Otras variantes

- GRASP extendido
- Búsqueda Tabú constructiva
- Algoritmos de construcción aleatorizados
- Algoritmos de inserción aleatoria



Artículos y videos en Poliformat

- **GRASP: Greedy Randomized Adaptive Search Procedures.** M. Resende, J.L. González. Inteligencia Artificial, [Inteligencia Artificial](#). No.19
- **Optimization by GRASP.** M. Resende, C. Ribeiro. Springer (2016).
- **Machine Learning into Metaheuristics.** E.Talbi ACM Computing Surveys (2021).
- **Greedy Randomized Adaptive Search Procedures.** T.A. Feo, M. Resende. J. of Global Optimization 6 (1995).



Búsqueda Heurística en IA: Constructiva

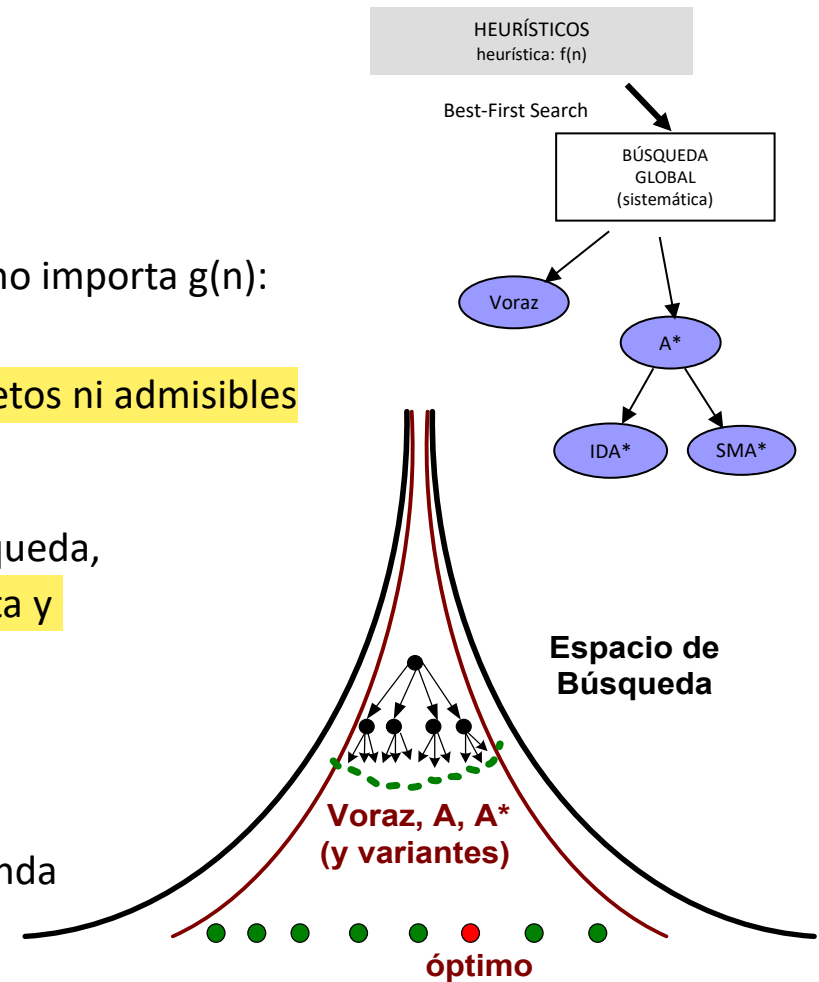
a) Búsqueda Global/Sistemática (típica)

Búsqueda Voraz (Greedy)

- Caso típico de “búsqueda el primero mejor”, donde no importa $g(n)$:
$$f(n) = h(n)$$
- No siempre encuentran solución o acaban. Ni completos ni admisibles

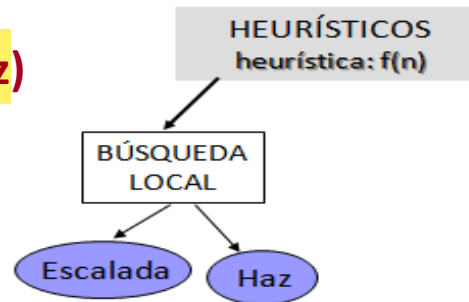
Algoritmo A

- Combinación de *búsqueda voraz* (reduce coste de búsqueda, pero no óptima ni completa) y *coste uniforme* (completa y óptima, pero ineficiente)
$$f(n) = g(n) + h(n)$$
- Algoritmo A*: $\forall n, h(n) \leq h^*(n)$ Completa y admisible
- Pero **requiere mucha memoria** (un algoritmo que expanda menos nodos que A* no tendrá admisibilidad)



b) Búsqueda Local (Escalada, Haz)

- Ciclos, Óptimos Locales, etc.



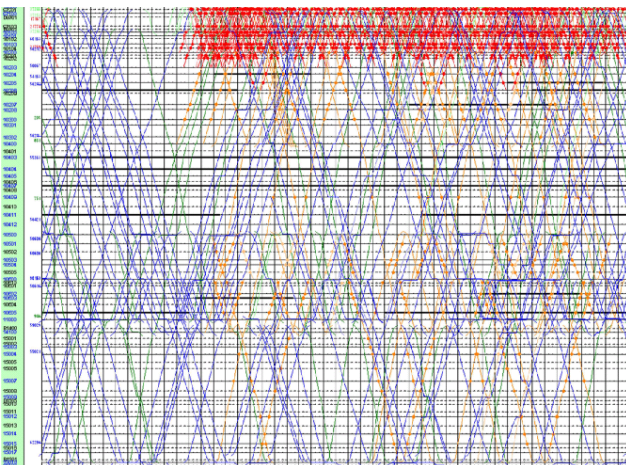
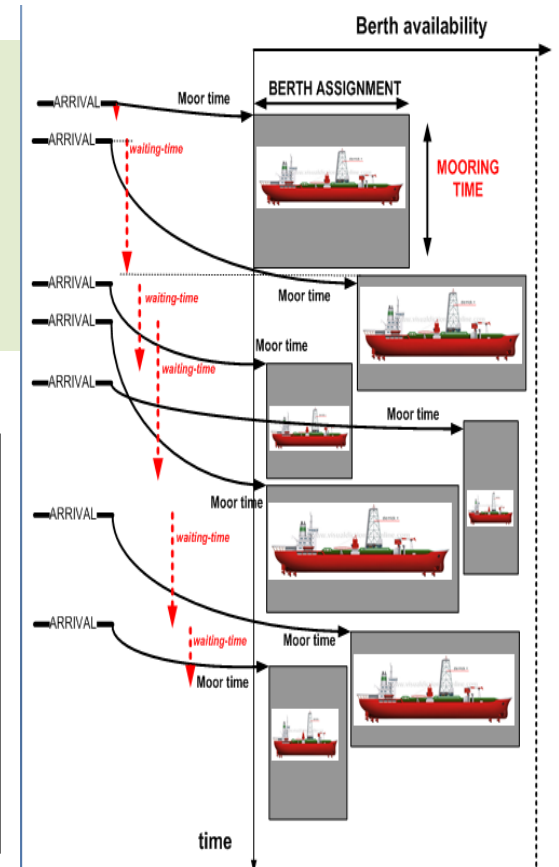
Métodos Constructivos:

Búsqueda de la solución en un espacio de estados: nuevos elementos *se añaden iterativamente a la solución*

Apropiados para

- Problemas en los que es difícil obtener soluciones iniciales (factibilidad)
- Vecindario muy amplio o poco estructurado (metaheurísticas de mejora)
- Complejidad de la representación de la solución (en metaheurísticas poblacionales)

JUNIO																																		
Cod.	Nombre	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	1	2	
		L	M	X	J	V	S	D	L	M	X	J	V	S	D	L	M	X	J	V	S	D	L	M	X	J	V	S	D	L	M	X	J	
 1		N	N	-	T	T	T	T	T	-	-	M	M	M	M	-	-	-	-	-	-	-	N	N	N	N	-	-	-	-	-	-	T	V
 2		-	M	M	M	M	-	-	-	-	-	-	N	N	N	N	-	-	-	-	T	T	T	T	-	N	N	N	N	-	-	-	V	T
 3		M	M	M	M	M	-	-	M	M	M	M	M	-	-	-	-	-	M	M	M	M	M	M	-	-	M	M	M	M	M	M	V	V
 4		M	M	M	M	M	-	-	-	N	N	-	-	-	-	T	T	T	T	T	-	-	-	-	N	N	N	N	N	N	-	-	-	M
 5		T	T	T	T	T	-	-	-	T	T	T	T	-	-	-	-	-	M	M	M	M	M	M	-	-	M	M	M	M	M	M	-	T
 6		N	N	-	-	-	-	T	T	T	T	T	-	-	-	N	N	N	N	-	-	-	-	-	M	M	M	M	M	M	-	-	T	T
 7		-	-	-	-	N	N	N	N	-	-	-	M	M	M	M	M	-	-	T	T	T	T	T	T	-	-	-	-	-	T	T	V	V
 8		-	-	-	-	N	N	N	N	-	-	T	T	T	T	T	T	-	-	N	N	N	N	N	N	-	-	-	-	-	T	T	T	T
 9		-	-	-	-	T	T	T	T	T	-	-	M	M	M	M	M	-	-	N	N	N	N	N	N	-	-	-	-	-	M	M	M	M

[illegible]

“A probabilistic heuristic for a computationally difficult set covering problem”

Metaheurística de **multi-inicio** (*multi-start*) o iterativo, donde cada **iteración** tiene dos fases:

(i) **Fase Constructiva:** se encuentra una solución factible, mediante una heurística constructiva

- No necesita una solución inicial
- La **longitud del camino** (solución) debe ser **limitada**
- Los **elementos a añadir a la solución** en cada paso son **limitados** (baja ramificación)

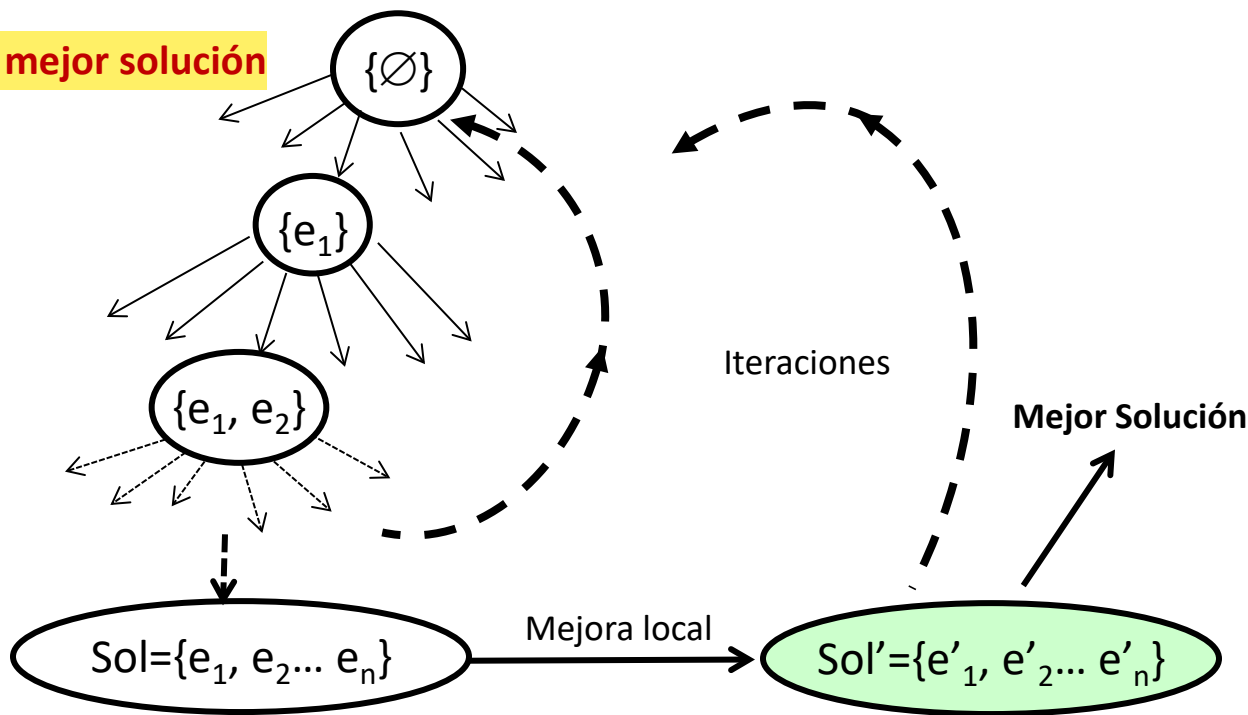
(ii) **Fase de Mejora con Búsqueda local:**

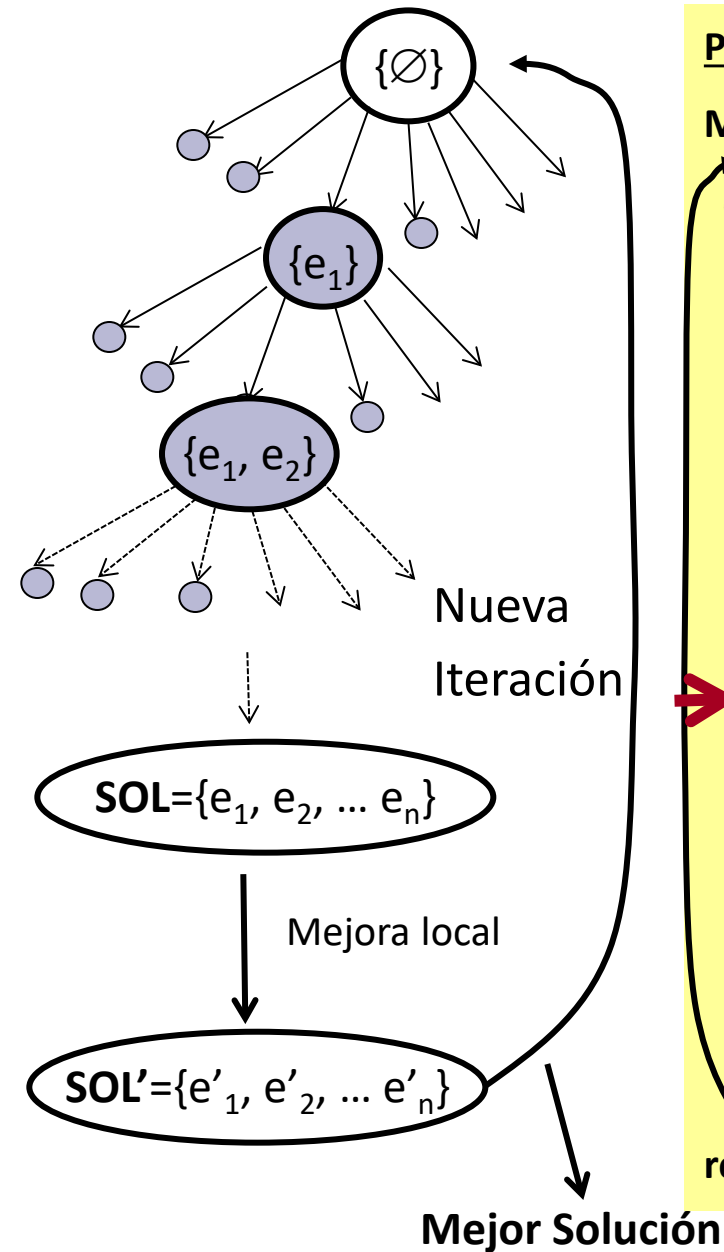
- La **vecindad** de la solución se explora para mejorarla, típicamente, mediante **escalada**: mejor candidato (*best-improvement*) o primer candidato (*first-improvement*)

Además, vamos actualizando la mejor solución

Aplicaciones típicas:
Planificación y Scheduling,
Construcción de
soluciones,
Path-finding, etc.

Aplicaciones
(2022)





Procedimiento GRASP

$SOL_{MEJOR} = \emptyset, \alpha, f_{coste}$

Mientras (no condición de parada, o k iteraciones)

a) Fase Constructiva: $SOL = \emptyset$

Greedy Randomized Construction (Semilla)

Iterativamente:

- ➔ Formar lista de elementos candidatos "C". Evaluar coste
- ➔ Formar lista restringida de los mejores candidatos (α y RCL)
- ➔ Seleccionar un elemento aleatoriamente de la lista restringida RCL
- Hasta encontrar solución (SOL)

➔ **b) Fase de Mejora:** $SOL' = \text{búsqueda local}(SOL)$

Proceso de búsqueda local a partir de la solución construida (SOL)

c) Actualización: $SOL_{MEJOR} = \text{actualiza}(SOL', SOL_{MEJOR})$

Si la solución obtenida mejora a la mejor almacenada, actualizarla

return (SOL_{MEJOR})

Fase Constructiva (greedy). Minimización

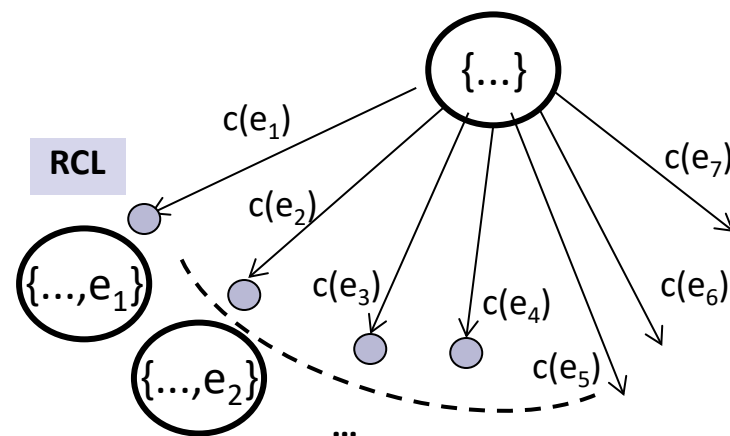
En cada iteración:

- **Elementos_Candidatos $C=\{e_i\}$:**

Conjunto de elementos factibles a incorporar a la solución parcial que se está construyendo

La incorporación de cada elemento de $\{e_i\}$ añade un **coste adicional** $c(e_i) \in [c^{\min}, c^{\max}]$

- **RCL:** Lista de Candidatos Restringida con los mejores elementos que menos empeoran la función de coste (uso de $\alpha \in [0,1]$)



En maximización: $c(e_i) \in [c^{\max} - \alpha(c^{\max} - c^{\min}), c^{\max}]$

La RCL está formada por aquellos elementos e_i , tal que $c(e_i) \in [c^{\min}, c^{\min} + \alpha(c^{\max} - c^{\min})]$

α sirve para diversificar la construcción y no repetir soluciones

$\alpha=0$ es un algoritmo completamente *greedy*; $\alpha=1$ es una estrategia aleatoria

- Elegir aleatoriamente un elemento de **RCL**

Fase Constructiva (greedy). Algoritmo general Minimización

```
procedimiento Greedy Randomized Construction(Semilla)
 $s \leftarrow 0$ 
Inicializa el conjunto candidato:  $C \leftarrow E$ 
Evalua el costo incremental  $c(e) \forall e \in C$ 
while  $C \neq \emptyset$  do
     $c^{min} \leftarrow \min\{c(e) | e \in C\}$ 
     $c^{max} \leftarrow \max\{c(e) | e \in C\}$ 
     $RCL \leftarrow \{e \in C | c(e) \leq c^{min} + \alpha(c^{max} - c^{min})\}$ 
    selecciona un elemento  $t$  de RCL aleatoriamente
     $s \leftarrow s \cup \{t\}$ 
    actualiza el conjunto candidato  $C$ 
    re-evalua los costos incrementales  $c(e) \forall e \in C$ 
return  $s$ 
```

- $c(e)$ representa el coste incremental de añadir “e” a la solución parcial
- En cada iteración **no** se selecciona al mejor candidato, sino un **elemento aleatorio** del conjunto de mejores candidatos (RCL)
- En el caso de que se violen **restricciones de factibilidad**, $c(e)=\infty$
- Coste incremental $c(e)$ puede ser obtenido como un valor heurístico $h(e)$

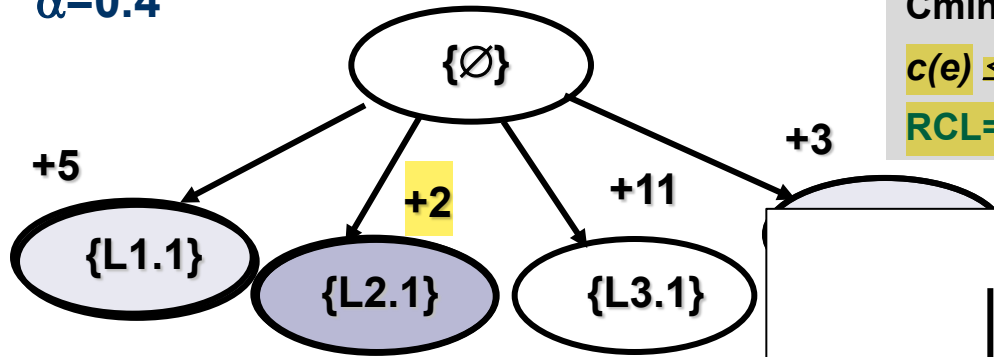
Ejemplo 1

4 lectores (L1, L2, L3, L4) desean leer tres periódicos (P1, P2, P3) en orden y en el mínimo tiempo en total

Greedy: Fase Constructiva
{Lector.Periódico}

$S = \{\emptyset\}; \alpha = 0.4$

	Ready-Time	P1	P2	P3
L1	0	5'	10'	2'
L2	0	2'	5'	5'
L3	0	11'	15'	15'
L4	0	3'	5'	5'

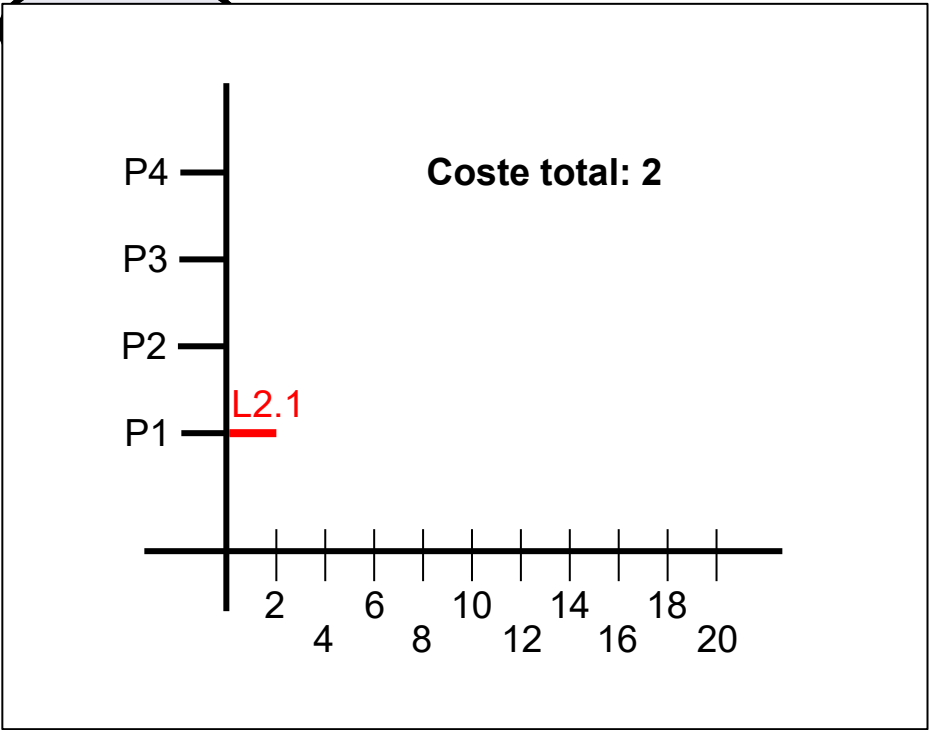


Cmin=2; Cmax=11

$c(e) \leq 2 + 0.4 (11-2) = 5.6$

Se descarta la L3.1 por coste +11

RCL= {L1.1, L2.1, L4.1}; S = {L2.1} (azar)



Ejemplo 1

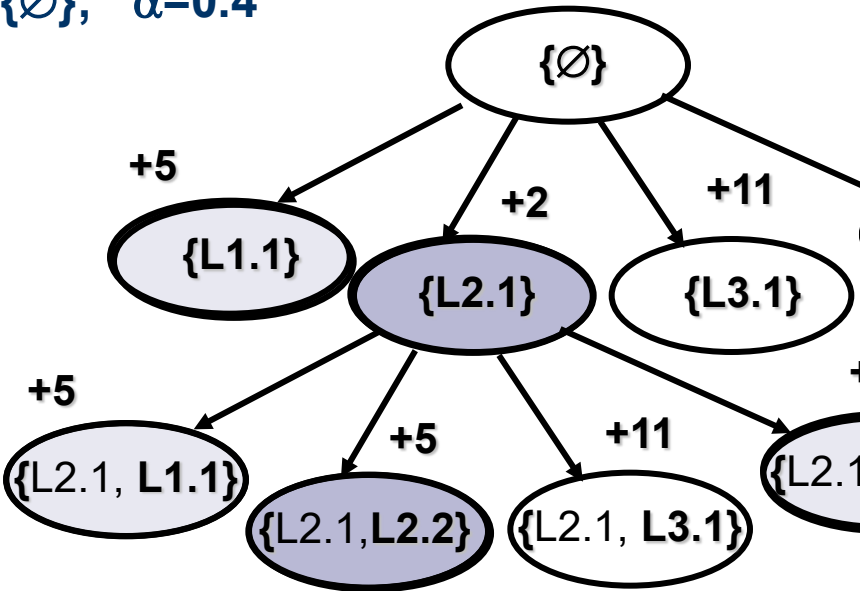
4 lectores (L1, L2, L3, L4) desean leer tres periódicos (P1, P2, P3) en orden y en el mínimo tiempo en total

Greedy: Fase Constructiva

{Lector.Periódico}

S={∅}; α=0.4

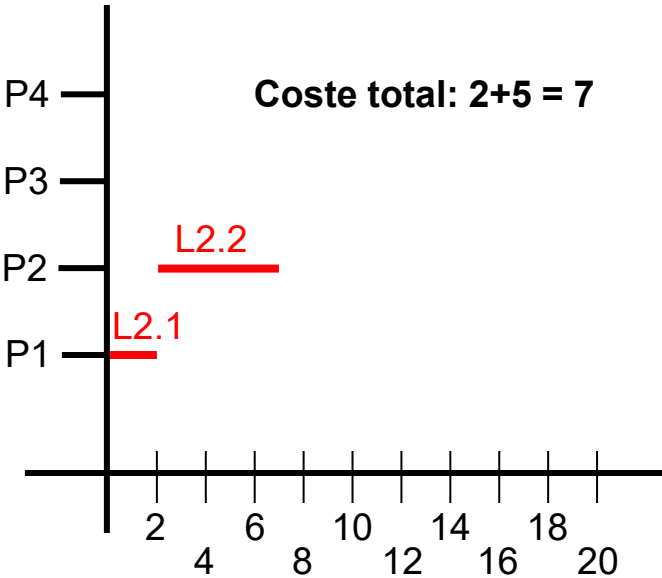
	Ready-Time	P1	P2	P3
L1	0	5'	10'	2'
L2	0	2'	5'	5'
L3	0	11'	15'	15'
L4	0	3'	5'	5'



Cmin=2; Cmax=11

$$c(e) \leq 2 + 0.4 (11-2) = 5.6$$

RCL= {L1.1, L2.1, L4.1}; S = {L2.1}



Ejemplo 1

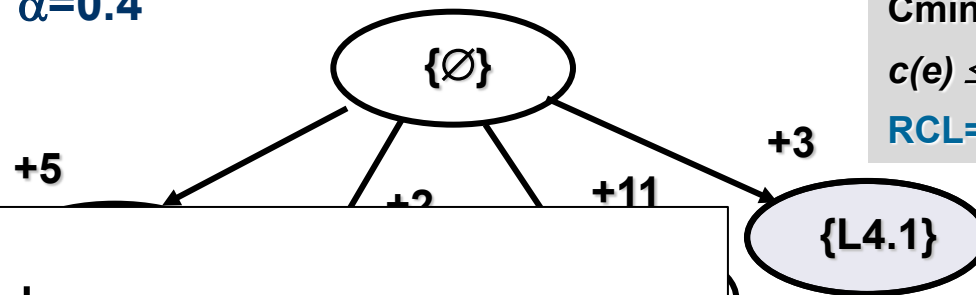
4 lectores (L1, L2, L3, L4) desean leer tres periódicos (P1, P2, P3) en orden y en el mínimo tiempo en total

Greedy: Fase Constructiva

{Lector.Periódico}

$S = \{\emptyset\}; \quad \alpha = 0.4$

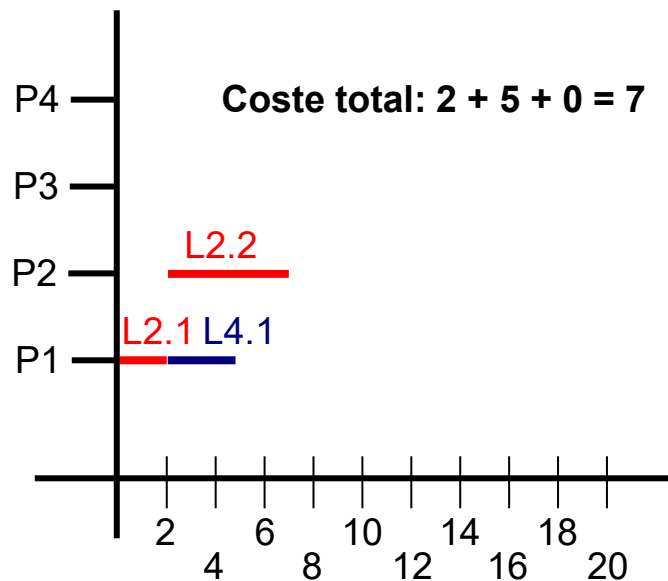
	Ready-Time	P1	P2	P3
L1	0	5'	10'	2'
L2	0	2'	5'	5'
L3	0	11'	15'	15'
L4	0	3'	5'	5'



Cmin=2; Cmax=11

$$c(e) \leq 2 + 0.4 (11-2) = 5.6$$

RCL= {L1.1, L2.1, L4.1}; S = {L2.1}



Cmin=3; Cmax=11

$$c(e) \leq 3 + 0.4 (11-3)=6.2$$

RCL= {L1.1, L2.2, L4.1}; S = {L2.1, L2.2}

Cmin=0; Cmax=6

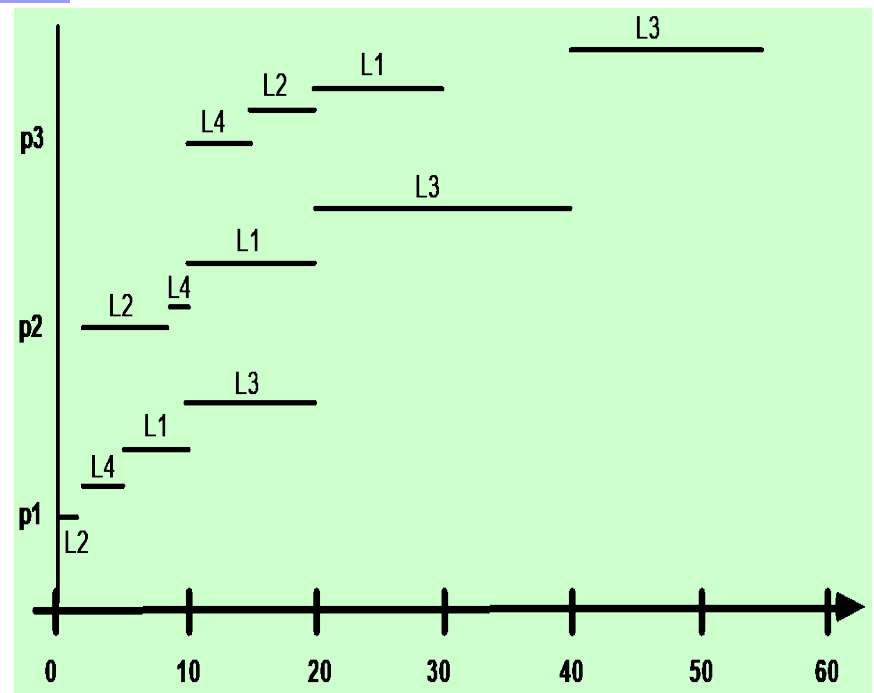
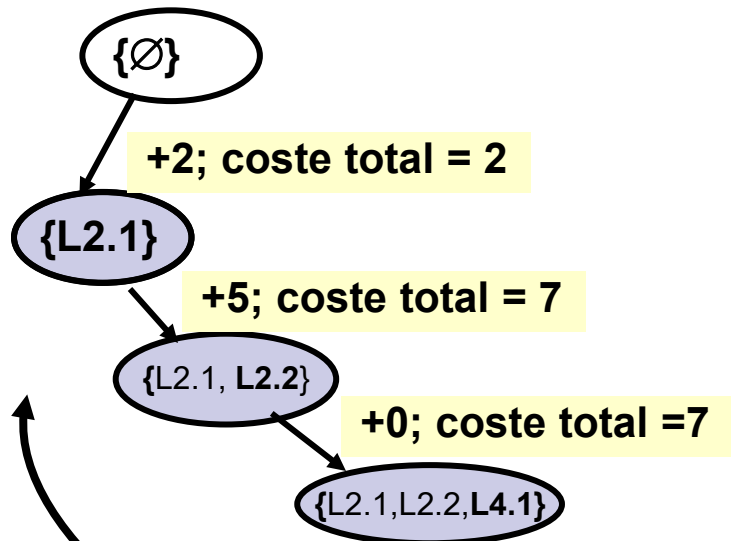
$$c(e) \leq 0 + 0.4 (6-0)=2.4$$

RCL= {L1.1, L4.1}; S = {L2.1, L2.2, L4.1}

L4.1}

Ejemplo 1. En resumen...

	Ready-Time	P1	P2	P3
L1	0	5'	10'	2'
L2	0	2'	5'	5'
L3	0	11'	15'	15'
L4	0	3'	5'	5'



Ejemplo 2. Realizar N obras, pidiendo presupuesto a M empresas

Ejemplo, URJC	Obra O1	Obra O2	Obra O3	Obra O4
EmpresaE1	2	3	2	4
EmpresaE2	5	5	4	5
EmpresaE3	6	5	4	3
EmpresaE4	10	8	6	6

Búsqueda (4!):
4x3x2x1= 24 nodos

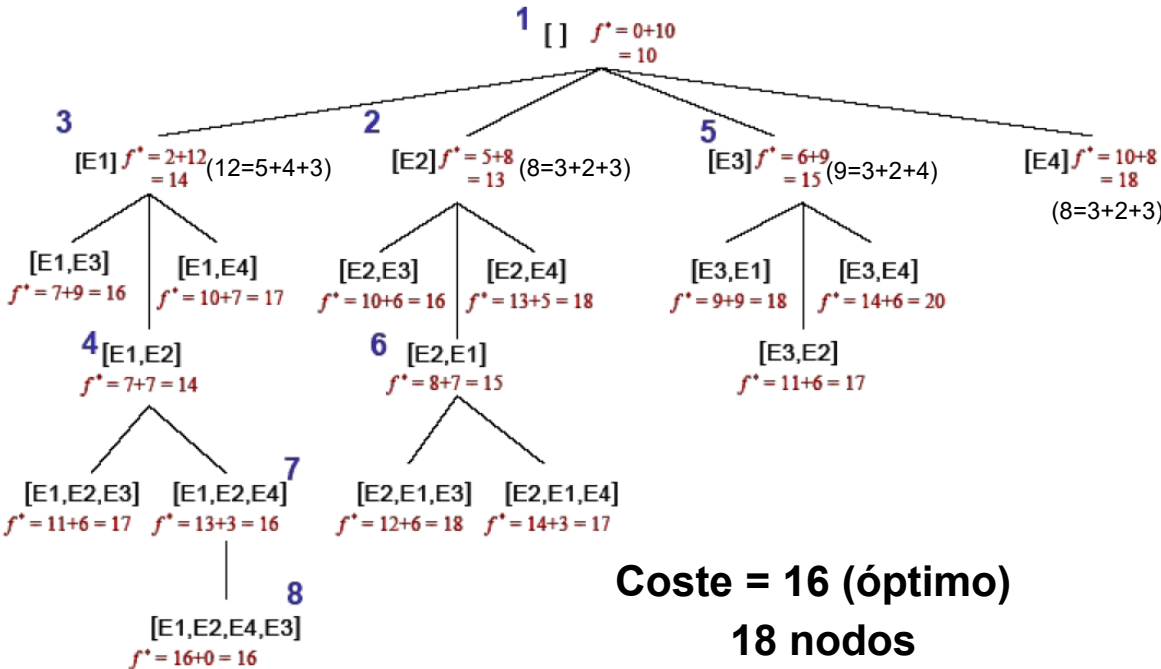
Se decide asignar **una sola obra a cada empresa**. El problema consiste en decidir qué obra se debe asignar a cada empresa, tal que se **minimice** el coste total.

Resolución mediante la aplicación Algoritmo A*

Cada nivel es una obra

$f(n)=g(n)+h^*(n)$

$h^*(n)$ =suma de los costes mínimos de las obras no asignadas, considerando solo las empresas todavía sin adjudicación



Ejercicio GRASP.

Realizar la Fase Constructiva de la técnica GRASP con los siguientes parámetros
 $S=\{\emptyset\}$; $\alpha=0.4$

La selección del individuo a escoger del RCL se deja a libertad de cada persona

Indicad el número de nodos generados y la calidad de la solución obtenida

¿Se consigue la solución óptima?

Ejemplo, URJC	Obra O1	Obra O2	Obra O3	Obra O4
EmpresaE1	2	3	2	4
EmpresaE2	5	5	4	5
EmpresaE3	6	5	4	3
EmpresaE4	10	8	6	6

GRASP. Importancia de la fase de Mejora Local

En la fase constructiva se obtienen soluciones optimizadas a partir de decisiones locales *aleatorizadas* de la RCL

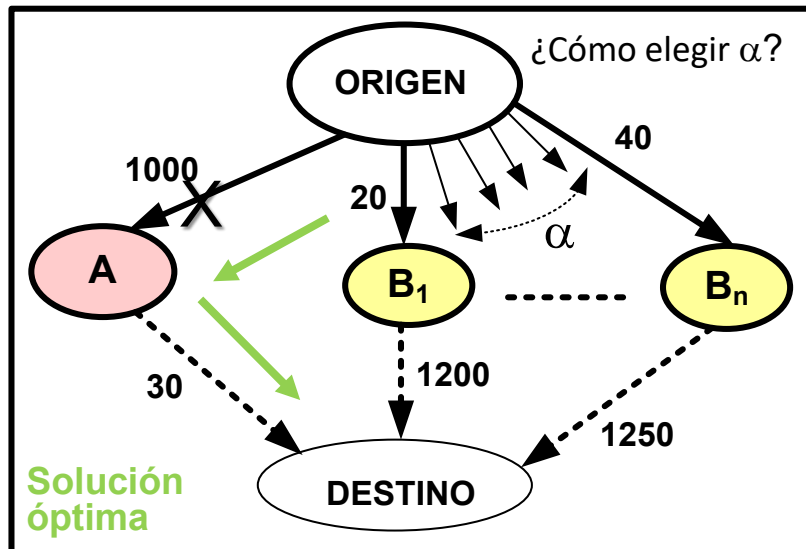
$$\text{RCL} \equiv \{c^{\min}, \quad c^{\min} + \alpha(c^{\max} - c^{\min})\}$$

Si $\alpha > 0$, se abre más la elección del sucesor, incrementando la exploración

Si $\alpha < 1$, la peor (o peores decisiones) en un cierto nivel **tenderán a ignorarse**

En el caso de que una *pésima* decisión en un cierto nivel conduzca a la solución óptima, es imposible que la fase constructiva de GRASP pueda encontrarla – solo se podrá encontrar en la fase de mejora

Ejemplo:



La solución óptima (o altamente optimizada) está compuesta de alguna decisión local donde el coste/heurística obtiene pésimas evaluaciones. Si α no es muy *acertado*, la fase constructiva está muy *limitada*

GRASP tenderá a a caer en óptimos locales, en sucesivas iteraciones, sin que le sea posible alcanzar el óptimo global – la fase de mejora es fundamental

Algoritmo GRASP extendido

La fase constructiva se **mantiene igual**

En la fase de mejora se **añade un operador de mutación** para tratar de mejorar todavía más la solución encontrada

Estructura básica algoritmo GRASP

- Mientras no se satisfaga el criterio de parada
 1. Construir una solución greedy aleatoria
 2. Aplicar una técnica de búsqueda local a la solución greedy aleatoria obtenida en el paso anterior (para mejorarla)
 3. Actualizar la mejor solución encontrada

Estructura algoritmo GRASP extendido

- Mientras no se satisfaga el criterio de parada
 - Solución = Solución greedy aleatoria
 - Repetir L veces
 - Solución = Búsqueda local (Solución)
 - Actualizar la mejor solución (si corresponde)
 - Solución = Mutación(Solución)



Cada elemento de la solución se muta (altera) con probabilidad p y se mantiene con probabilidad $1-p$

Otras variantes

Semi-greedy

- No tiene una fase de mejora local de la solución (solo constructivo).
- Peligro de perder buenas soluciones si requiere un peor paso local.

Selección RCL

- a) **Reactive GRASP:** Los ajustes de la RCL (α) se determinan dinámicamente según el estado de la búsqueda ($\approx$ enfriamiento simulado). Se inicia con $\alpha \approx 1$ y se va disminuyendo.
- b) **Selección $e \in RCL$ no aleatoria:** Elitista, Ruleta, Rango, etc.

Clustering-GRASP, Path Relinking, etc.

- Se guardan las mejores soluciones (no solo la mejor) de los reinicios. Las metaheurísticas de mejora se inician desde diversas mejores soluciones previas
- No formar la RCL entre sucesores inmediatos del estado actual, sino los de un nivel siguiente, etc.

Búsqueda Tabú constructiva

- Se construye una solución paso a paso, agregando elementos a una solución parcial, evitando ciclos y movimientos poco efectivos mediante el uso de la lista tabú

En general, cuando hablamos de métodos constructivos

Algoritmos de Construcción Aleatorizados

- Construcción iterativa de soluciones. Se van agregando elementos uno a uno.
- Utilizan aleatoriedad para explorar el espacio de soluciones (no deterministas para obtener mayor diversificación).
- Utilización de heurísticas para equilibrar exploración y explotación.
- Opcionalmente se puede llevar a cabo una mejora de la solución mediante búsqueda local.
- Amplia exploración y flexible, pero sin garantía de optimalidad.

Algoritmos de Inserción Aleatoria

- Destinados a problemas donde la solución implica algún tipo de secuencia.
- Se parte de una solución vacía o incompleta y se van añadiendo elementos uno a uno en posiciones específicas que optimicen algún criterio de calidad.
- Inserción determinista o aleatoria. Ej.: inserción simple (la que mejore la métrica), inserción aleatoria (diversificar exploración), inserción heurística (dependiente del problema), etc.
- Algoritmos simples y flexibles, pero sin garantía de optimalidad.

Reshuffling a Container Yard with 6 rows x 6 stacks (maximum high: 4 containers):

To improve the planning performance

- uses a domain-dependent heuristic
- the next action is randomly chosen from the actions that obtain a better performance of the heuristic function

Algorithm 1: Pseudo-code of the domain-dependent heuristic function

Data: s : state to evaluate

Result: h , heuristic value of s

$h = 0$;

if $\exists x$ - container / holding(x) $\in s$ then

 if goal-container(x) then

$h = 0.1$;

 else

$h = 0.5$;

 end

end

for each row r in the yard-bay do

$\Delta h = 0$;

 for x - container / at(x , r) $\in s \wedge$ goal-container(x) do

 if $\nexists y$ - container / goal-container(y) $\wedge$ on(y , x) $\in s$ then

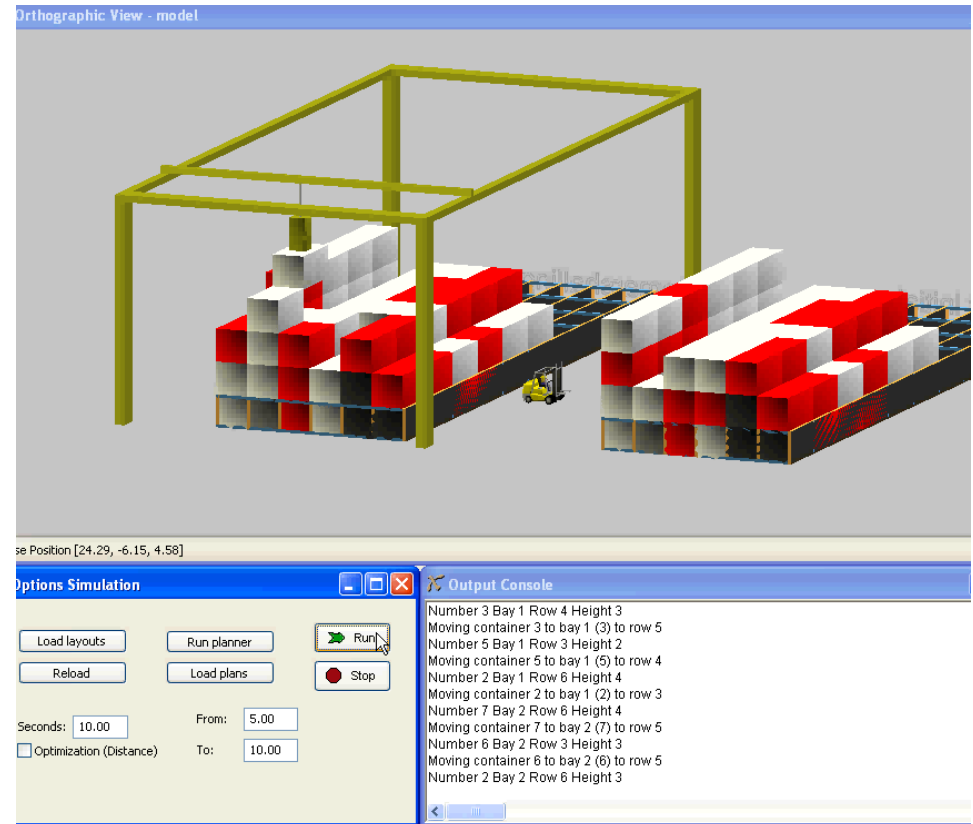
$\Delta h = \max(\Delta h, \text{numContainersOn}(x))$;

 end

 end

$h += \Delta h$;

end



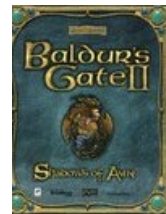
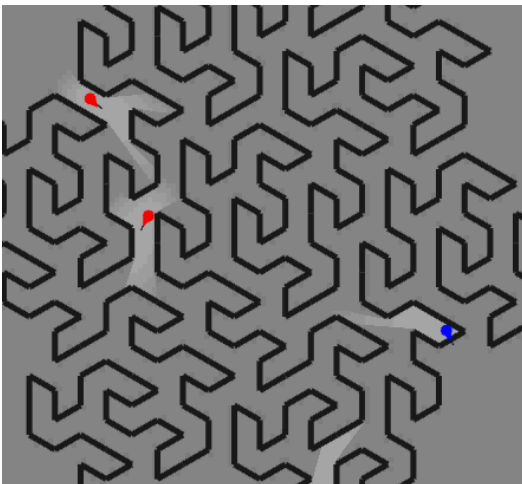
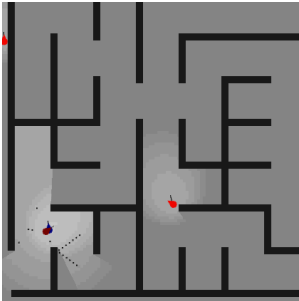
Ejemplo de aplicación: Path-finding

Path-finding: Problema de encontrar el mejor camino/ruta (habitualmente, el más corto) entre dos puntos a través de un mapa con obstáculos.

Problemas de laberinto, Aplicaciones para videojuegos, etc.

Algoritmos típicos: Dijkstra (muy costoso en laberintos complejos),
Algoritmos A (equivalente a Dijkstra, cuando $h(n)=0$),
A* sobre-informados, etc.

Metaheurísticas: Grasp, Inteligencia de Enjambre, Algoritmo de las hormigas...



<http://www.movingai.com/benchmarks/>

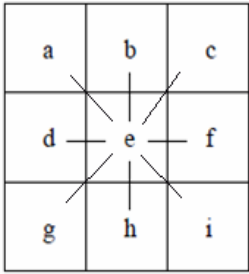
Benchmarks for Grid-Based Path finding.

IEEE Trans. on Comp. Int. and AI in games

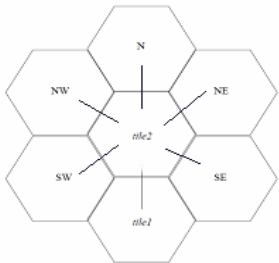
Grid-based path-finding: Descomposición del Mapa del terreno en Rejillas



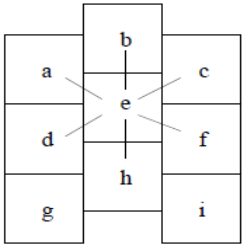
Alternativas de Descomposición:



Octiles

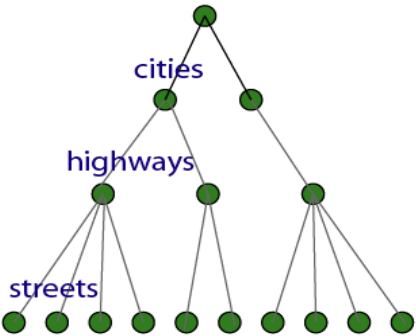


Hexes

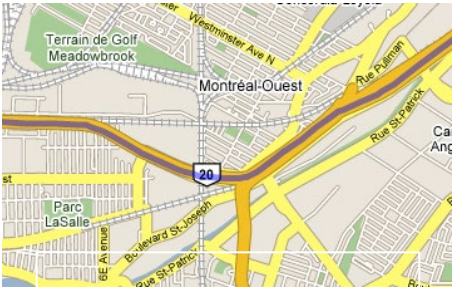


Texes

- A mayor nivel de descomposición, mayor nivel de ramificación.
- Técnica Jerárquica: Path-finding jerárquico



Paso 1) Búsqueda Conexiones Ciudades



Paso 2) Búsqueda de Calle

Conclusiones

- No siempre disponemos de una solución de partida. Hay veces que hay que construirla desde cero → **metaheurísticas constructivas**
- Las metaheurísticas constructivas combinan construcción + mejora
 - Generan soluciones de forma incremental hasta obtener una solución completa
 - Las soluciones se pueden construir de distintas formas (voraz, aleatoriamente, etc.)
 - Los sucesores a incluir en RCL se eligen de un conjunto candidato (que se puede restringir en base a un parámetro α , o porque se define un tamaño limitado en la RCL)
 - El proceso de mejora puede hacerse de distinta forma: búsqueda local, aplicación de mutaciones, inserción de elementos, etc.
 - Aunque los criterios locales **no necesariamente** conducen a la solución óptima
- **Múltiples variantes.** Técnicas rápidas que se pueden utilizar como punto de partida para otras técnicas de optimización.

Ejercicio

Exercise Given the following instance of the *Parallel Machine Scheduling Problem (PMSP)* with 3 machines:

t	a	b	c	d	e	f
d_t	27	12	15	25	11	16

a) apply a basic greedy heuristic in which the selection criterium is split into two steps:

- 1. select the task with the maximum time length: $i^* := \arg \max_{t \in T} d_t$;
- 2. select the machine that minimises the objective function: $m^* := \arg \min_{m \in M} f(x \cup \{t, m\})$.

In case of ties, in both steps choose the item of minimum index.

b) Apply a *GRASP* metaheuristic with a *Restricted Candidate List (RCL)* of two elements, assuming the following sequence of pseudorandom numbers: $r = (0.2, 0.2, 0.3, 0.8, 0.6, 0.1)$.

En lugar de usar un α ,
se limita el tamaño de
la RCL

Ejemplo de solución: M1={a, e}, M2={b,d} y M3={c,f},
con tiempos de procesamiento: M1=38, M2=37 y M3=31.
Tiempo total de procesamiento $f(x)=38$ (dado por M1)